

Simple Linear Regression – Marketing ROI Analysis

Project Overview:

In this project, a marketing dataset will be analyzed using Python and statsmodels to build a Simple Linear Regression model. This will be done by preparing data, selecting predictive variables, implementing OLS regression, validating model assumptions, and interpreting statistical outputs to drive business decisions. This project aims to show how data science is used to translate raw data into actionable marketing insights.

```
from google.colab import drive
drive.mount('/content/drive')
```

Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).

Simple Linear Regression – Marketing ROI Analysis Project Overview In this project, you will analyze a marketing dataset using Python and statsmodels to build a Simple Linear Regression model. You will learn how to prepare data, select predictive variables, implement OLS regression, validate model assumptions, and interpret statistical outputs to drive business decisions. This project helps you understand how data science translates raw data into actionable marketing insights.

Step-by-Step Project Tasks

- Load and explore the dataset; handle missing values
- Perform exploratory data analysis (EDA) with visualizations
- Identify the independent variable (TV, Radio, or Social Media) most correlated with Sales
- Build an OLS regression model using statsmodels
- Create diagnostic plots to test Linearity, Normality, and Homoscedasticity
- Interpret R-squared, coefficients, and p-values in business context
- Formulate a clear ROI-based recommendation for marketing budget allocation

Project Goal By the end of this project, your analysis should:

- Successfully load and clean the marketing dataset
- Visualize relationships and justify variable selection
- Fit a statistically valid Simple Linear Regression model
- Verify key regression assumptions through diagnostic plots
- Explain model results clearly to a non-technical stakeholder
- Recommend the marketing channel with the strongest ROI impact

Project Submission GitHub repository link containing:

- Jupyter Notebook (e.g., regression_analysis.ipynb) with all cells executed
- README.md with project overview and environment setup instructions (e.g., pip install pandas seaborn statsmodels)
- Clean, well-commented code with Markdown explanations of your analytical reasoning

1.1 Importing and Cleaning Data

```
# importing libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import plotly.express as px
import statsmodels.api as sm
```

```
# Importing data
df_sales = pd.read_csv("marketing_and_sales_data.csv")
df_sales.shape
df_sales.info()
df_sales.head()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 4572 entries, 0 to 4571
Data columns (total 4 columns):
#   Column          Non-Null Count  Dtype
---  ---
0   TV               4562 non-null   float64
1   Radio            4568 non-null   float64
2   Social_Media     4566 non-null   float64
3   Sales            4566 non-null   float64
dtypes: float64(4)
memory usage: 143.0 KB
```

	TV	Radio	Social_Media	Sales
0	16.0	6.566231	2.907983	54.732757
1	13.0	9.237765	2.409567	46.677897
2	41.0	15.886446	2.913410	150.177829
3	83.0	30.020028	6.922304	298.246340
4	15.0	8.437408	1.405998	56.594181

Next steps: [Generate code with df_sales](#) [New interactive sheet](#)

```
#Checking the data types of the data in the dataset
df_sales.select_dtypes(include="number")
```

	TV	Radio	Social_Media	Sales
0	16.0	6.566231	2.907983	54.732757
1	13.0	9.237765	2.409567	46.677897
2	41.0	15.886446	2.913410	150.177829
3	83.0	30.020028	6.922304	298.246340
4	15.0	8.437408	1.405998	56.594181
...	...	...	...	...
4567	26.0	4.472360	0.717090	94.685866
4568	71.0	20.610685	6.545573	249.101915
4569	44.0	19.800072	5.096192	163.631457
4570	71.0	17.534640	1.940873	253.610411
4571	42.0	15.966688	5.046548	148.202414

4572 rows × 4 columns

```
df_sales.select_dtypes(exclude="number")
```

0
1
2
3
4
...
4567
4568
4569
4570
4571

4572 rows × 0 columns

```
# Checking for null values
df_sales.isnull().sum()
```

	0
TV	10
Radio	4
Social_Media	6
Sales	6

dtype: int64

```
#Drop null values
df_sales.dropna(axis=0, inplace=True)
df_sales.head()
```

	TV	Radio	Social_Media	Sales
0	16.0	6.566231	2.907983	54.732757
1	13.0	9.237765	2.409567	46.677897
2	41.0	15.886446	2.913410	150.177829
3	83.0	30.020028	6.922304	298.246340
4	15.0	8.437408	1.405998	56.594181

Next steps: [Generate code with df_sales](#) [New interactive sheet](#)

```
#Checking for duplicates
print(df_sales.duplicated().value_counts())
print(df_sales["TV"].duplicated().value_counts())
print(df_sales["Radio"].duplicated().value_counts())
print(df_sales["Social_Media"].duplicated().value_counts())
print(df_sales["Sales"].duplicated().value_counts())
```

```
False    4546
Name: count, dtype: int64
TV
True      4455
False      91
Name: count, dtype: int64
Radio
False    4546
Name: count, dtype: int64
Social_Media
False    4546
Name: count, dtype: int64
Sales
False    4546
Name: count, dtype: int64
```

```
#Counting values
df_sales["TV"].value_counts()
```

```

count
TV
43.0    70
40.0    67
47.0    65
34.0    64
21.0    64
...     ...
76.0    36
81.0    35
91.0    34
71.0    32
57.0    31

91 rows x 1 columns

dtype: int64

```

```
df_sales.describe()
```

	TV	Radio	Social_Media	Sales
count	4546.000000	4546.000000	4546.000000	4546.000000
mean	54.062912	18.157533	3.323473	192.413332
std	26.104942	9.663260	2.211254	93.019873
min	10.000000	0.000684	0.000031	31.199409
25%	32.000000	10.555355	1.530822	112.434612
50%	53.000000	17.859513	3.055565	188.963678
75%	77.000000	25.640603	4.804919	272.324236
max	100.000000	48.871161	13.981662	364.079751

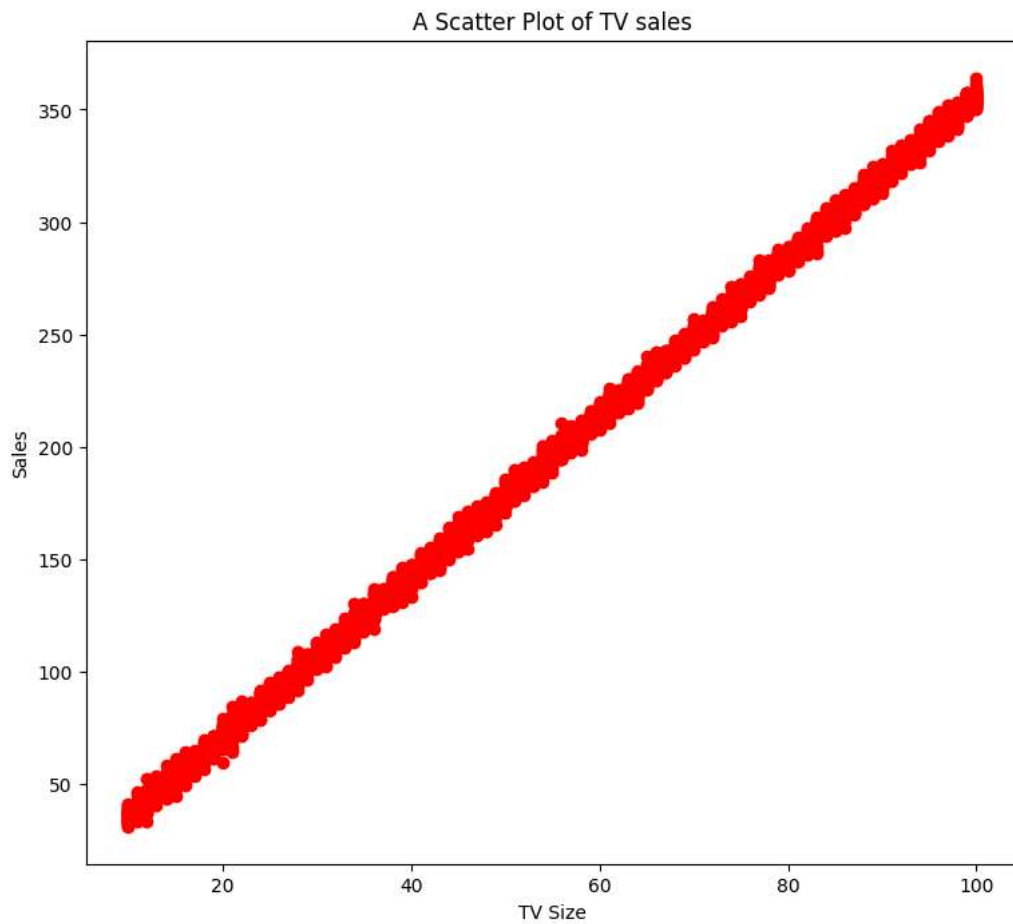
1.2. Exploratory Data Analysis (EDA)

```

fig, ax = plt.subplots(figsize=(9, 8))
ax.scatter(x=df_sales["TV"], y=df_sales["Sales"], color="red")
plt.title("A Scatter Plot of TV sales")
plt.xlabel("TV Size")
plt.ylabel("Sales")

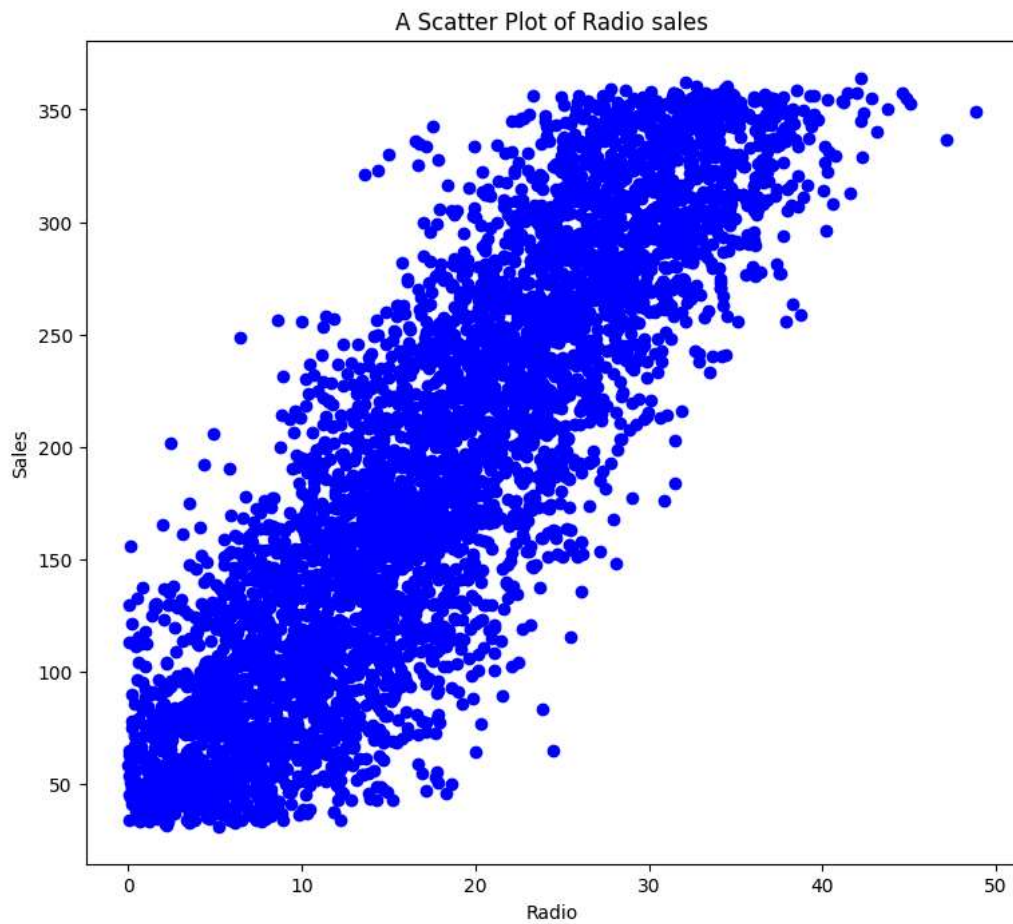
```

Text(0, 0.5, 'Sales')



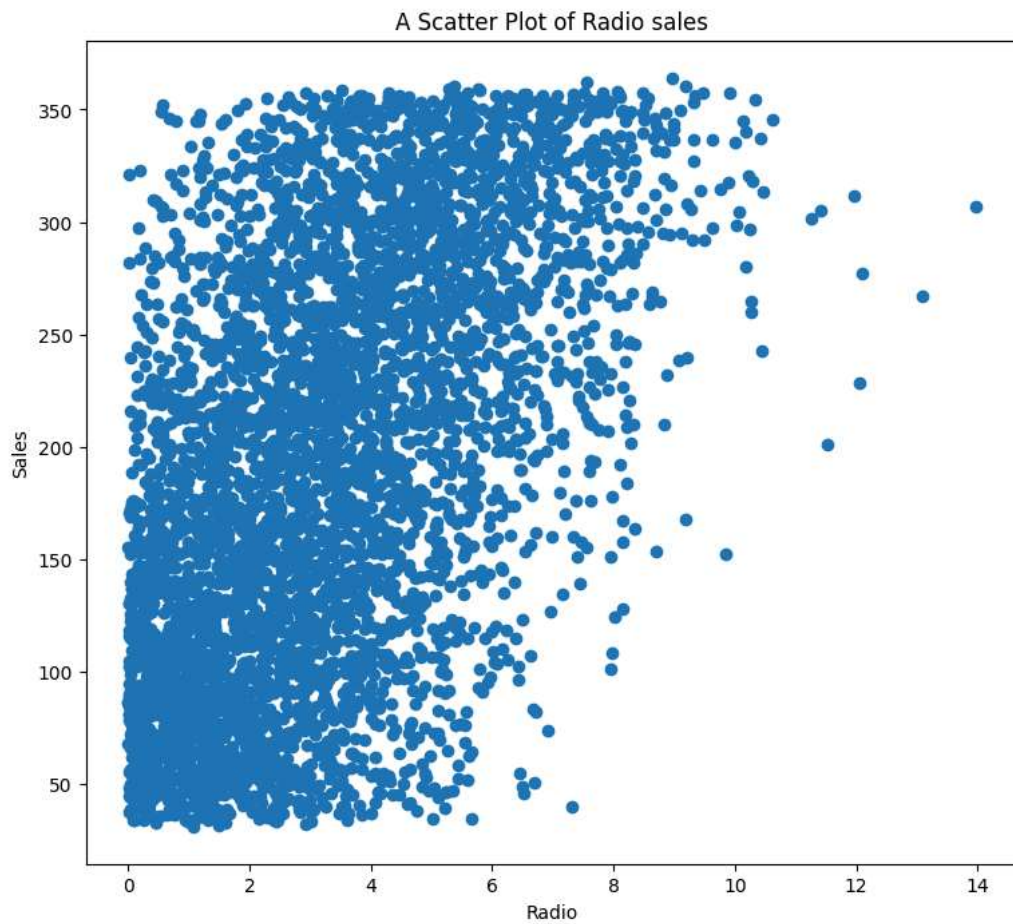
```
fig, ax = plt.subplots(figsize=(9, 8))
ax.scatter(x=df_sales["Radio"], y=df_sales["Sales"], color="blue")
plt.title("A Scatter Plot of Radio sales")
plt.xlabel("Radio")
plt.ylabel("Sales")
```

Text(0, 0.5, 'Sales')



```
fig, ax = plt.subplots(figsize=(9, 8))
ax.scatter(x=df_sales["Social_Media"], y=df_sales["Sales"])
plt.title("A Scatter Plot of Radio sales")
plt.xlabel("Radio")
plt.ylabel("Sales")
```

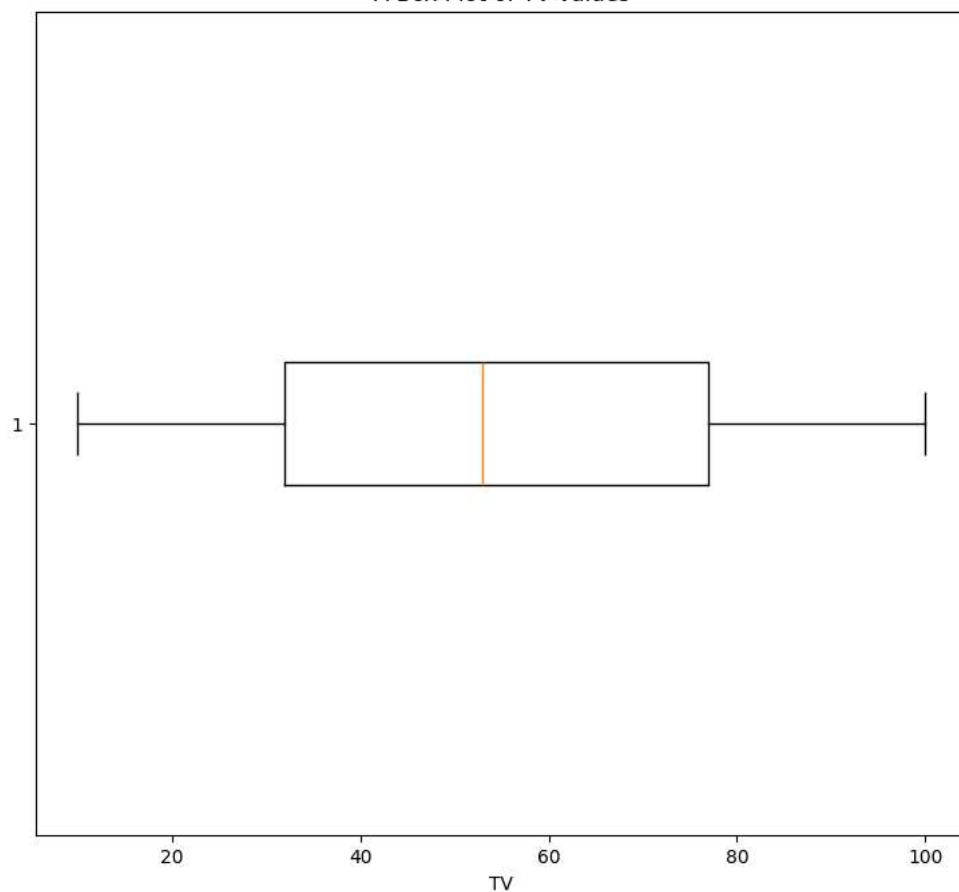
Text(0, 0.5, 'Sales')



```
fig, ax = plt.subplots(figsize=(9, 8))
ax.boxplot(df_sales["TV"], vert=False)
plt.title("A Box Plot of TV values")
plt.xlabel("TV")
```

```
Text(0.5, 0, 'TV')
```

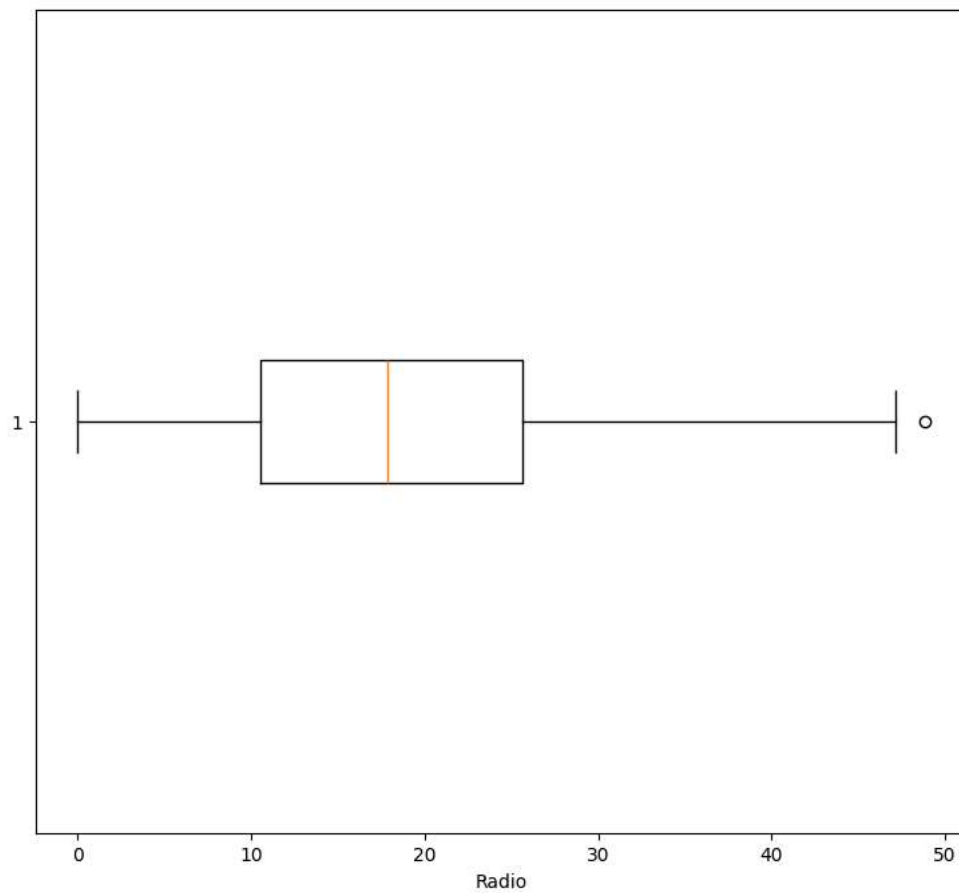
A Box Plot of TV values



```
fig, ax = plt.subplots(figsize=(9, 8))
ax.boxplot(df_sales["Radio"], vert=False)
plt.title("A Box Plot of Radio")
plt.xlabel("Radio")
```


Text(0.5, 0, 'Radio')

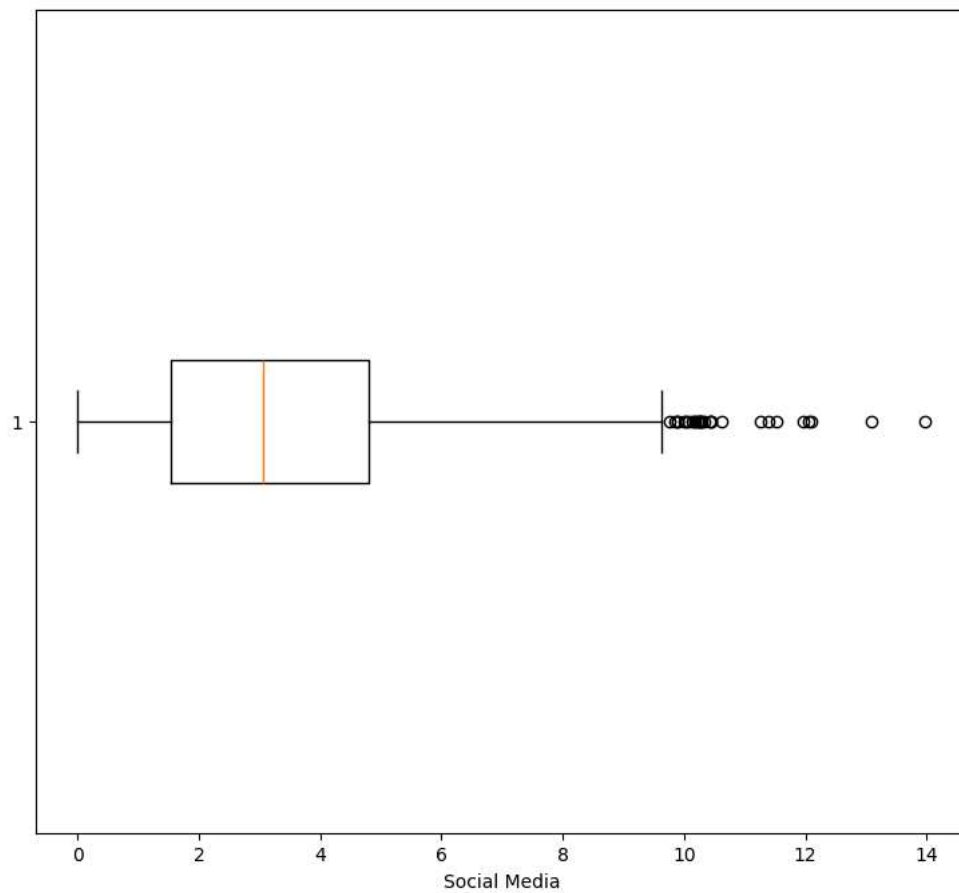
A Box Plot of Radio



```
fig, ax = plt.subplots(figsize=(9, 8))
ax.boxplot(df_sales["Social_Media"], vert=False)
plt.title("A Box Plot of Social Media Values")
plt.xlabel("Social Media")
```

Text(0.5, 0, 'Social Media')

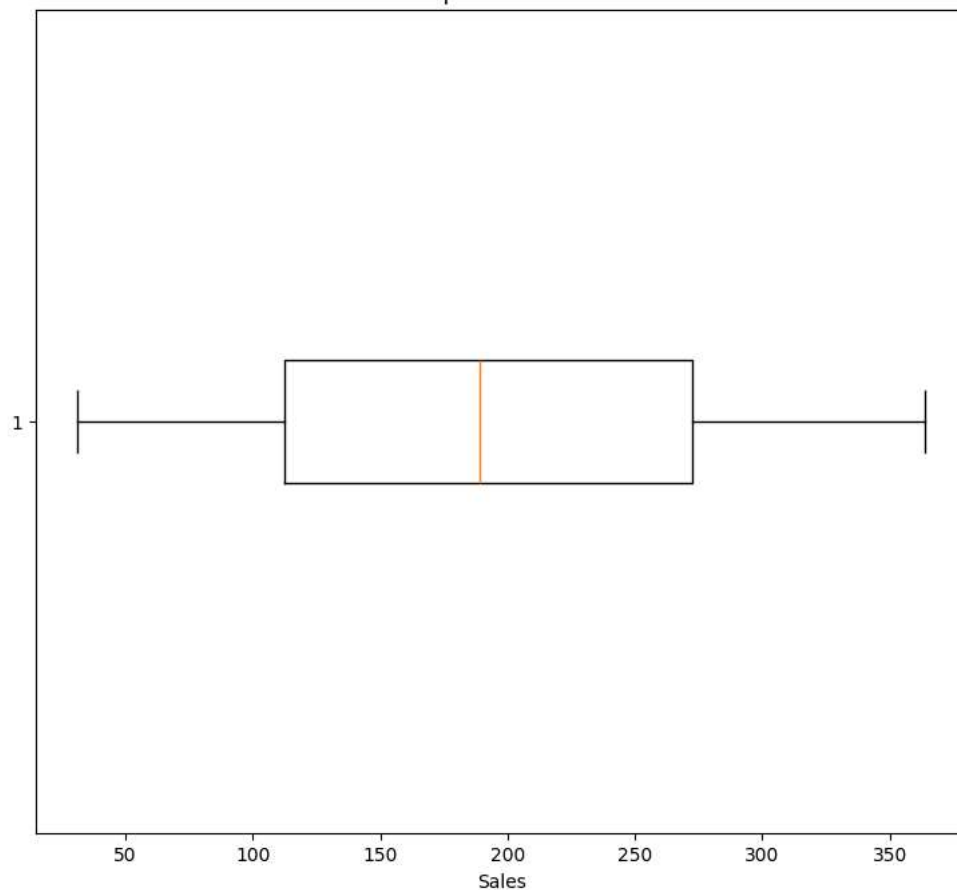
A Box Plot of Social Media Values



```
fig, ax = plt.subplots(figsize=(9, 8))
ax.boxplot(df_sales["Sales"], vert=False)
plt.title("A boxplot of the sales")
plt.xlabel("Sales")
```

```
Text(0.5, 0, 'Sales')
```

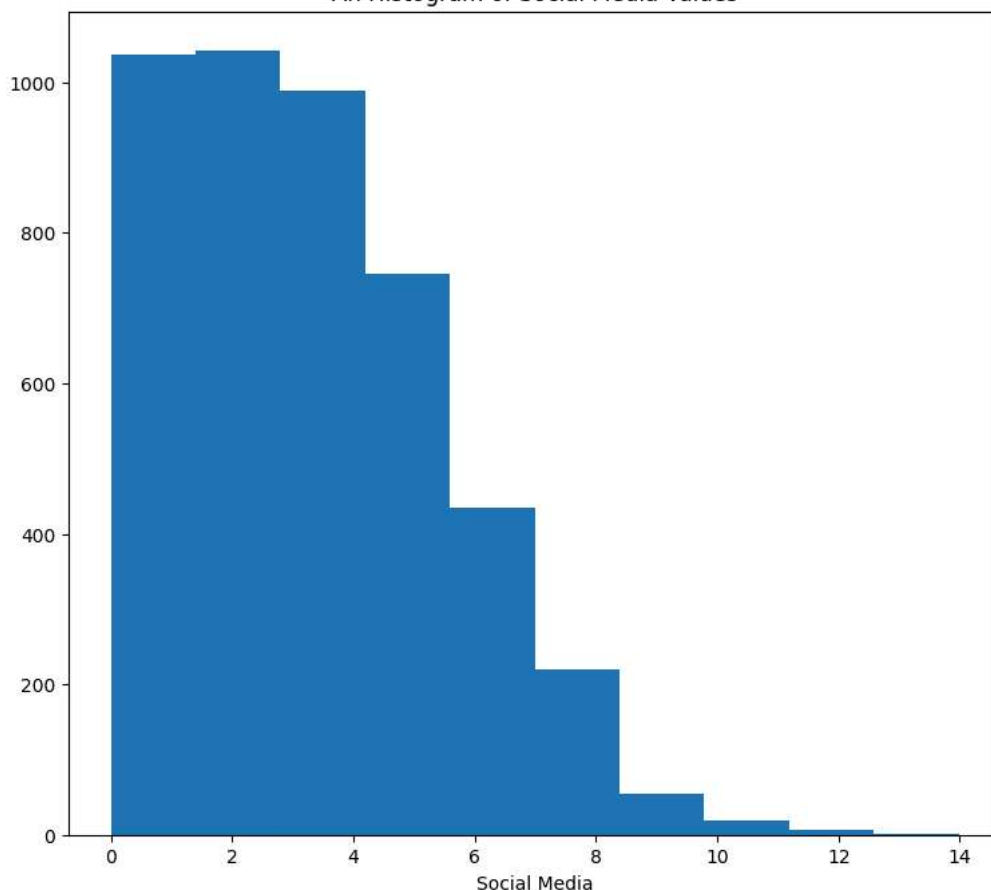
A boxplot of the sales



```
fig, ax = plt.subplots(figsize=(9, 8))
ax.hist(df_sales["Social_Media"])
plt.title("An Histogram of Social Media Values")
plt.xlabel("Social Media")
```

Text(0.5, 0, 'Social Media')

An Histogram of Social Media Values



The box plots and histogram indicate that there might be outliers in the Social media values. Further investigations will be carried out using Quantiles

```
# Calculating the Quantile
Q1 = df_sales.quantile(0.25)
Q3 = df_sales.quantile(0.75)
Qint = Q3 - Q1
#computing the range outside of which Outliers are defined
lower_bound = Q1 - 1.5*Qint
upper_bound = Q3 + 1.5*Qint

#converting the bound series to dictionaries
lower_bound_dict = lower_bound.to_dict()
upper_bound_dict = upper_bound.to_dict()
```

Handling Outliers

```
#Removing outliers lower than the lower bounds
for key in lower_bound_dict.keys():
    if (df_sales[key] < lower_bound_dict[key]).any():
        df_lower_outliers = df_sales[df_sales[key] < lower_bound_dict[key]]
        #printing outliers found
        print(df_lower_outliers)
        print(df_sales)
        #removing outliers with Pandas filtering
        df_sales = df_sales[df_sales[key] >= lower_bound_dict[key]]
    else:
        df_sales = df_sales

# Removing outliers greater than the upper bounds
for key in lower_bound_dict.keys():
    if (df_sales[key] > upper_bound_dict[key]).any():
```

```
df_upper_outliers = df_sales[df_sales[key] > upper_bound_dict[key]]
#printing outliers found
print(df_upper_outliers)
#removing outliers with masking/filtering
df_sales = df_sales[df_sales[key] <= upper_bound_dict[key]]
else:
    df_sales = df_sales
```

	TV	Radio	Social_Media	Sales
4331	99.0	48.871161	8.292208	349.156031
	TV	Radio	Social_Media	Sales
140	95.0	36.422306	9.997472	335.301431
212	100.0	32.920194	9.904286	357.124573
468	67.0	27.533561	10.453921	242.587371
504	85.0	36.077756	11.403625	304.970625
558	95.0	43.129332	10.174983	339.871732
603	73.0	31.327127	10.260554	259.859083
867	74.0	30.967507	13.083957	266.757198
1235	94.0	39.221198	10.420853	336.888405
1363	90.0	36.551512	9.883176	317.576616
1412	84.0	34.984570	11.260430	301.493120
1468	83.0	32.790106	10.243735	296.812238
1872	86.0	33.250050	10.063746	304.428485
1929	75.0	29.621449	10.260188	264.485251
2050	100.0	34.284714	10.337978	354.680451
2091	78.0	37.477608	12.108017	277.337551
2096	57.0	21.666267	11.534626	200.805601
2295	88.0	37.287223	11.964333	311.773774
2346	86.0	38.575779	13.981662	306.807598
2418	97.0	35.151475	10.610895	345.506826
2772	83.0	34.685101	10.030244	298.639410
3094	91.0	38.556143	10.214807	320.683149
3302	41.0	24.740521	9.855091	152.510022
3500	65.0	25.285103	12.054749	228.454384
3786	88.0	24.628032	9.751647	314.492417
3801	78.0	30.880488	10.188755	280.191966
4178	89.0	36.890920	10.297643	317.866489
4279	88.0	33.122470	10.455109	313.341447
4375	96.0	30.852798	10.127667	344.656524

df_sales

	TV	Radio	Social_Media	Sales	
0	16.0	6.566231	2.907983	54.732757	
1	13.0	9.237765	2.409567	46.677897	
2	41.0	15.886446	2.913410	150.177829	
3	83.0	30.020028	6.922304	298.246340	
4	15.0	8.437408	1.405998	56.594181	
...	...	...	...	...	
4567	26.0	4.472360	0.717090	94.685866	
4568	71.0	20.610685	6.545573	249.101915	
4569	44.0	19.800072	5.096192	163.631457	
4570	71.0	17.534640	1.940873	253.610411	
4571	42.0	15.966688	5.046548	148.202414	

4517 rows × 4 columns

Next steps: [Generate code with df_sales](#) [New interactive sheet](#)

Investigating Multicollinearity

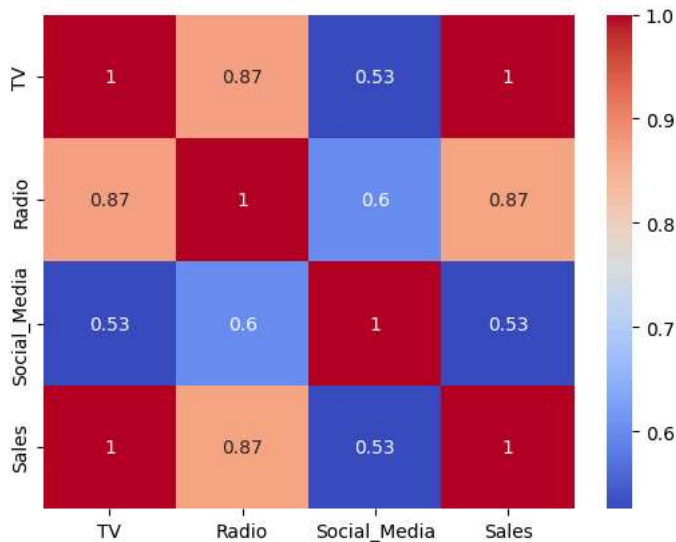
```
df_sales_corr = df_sales.corr(method="pearson", numeric_only=True)
df_sales_corr
```

	TV	Radio	Social_Media	Sales
TV	1.000000	0.868617	0.525867	0.999495
Radio	0.868617	1.000000	0.599865	0.868076
Social_Media	0.525867	0.599865	1.000000	0.525521
Sales	0.999495	0.868076	0.525521	1.000000

Next steps: [Generate code with df_sales_corr](#)[New interactive sheet](#)

```
sns.heatmap(df_sales_corr, annot=True, cmap="coolwarm")
```

<Axes: >



After the analysis, we can see that Radio and TV have strong correlations. In addition, TV has stronger correlations with Sales, hence we can drop TV

```
df_sales.drop(columns="Radio", inplace=True)
```

df_sales

	TV	Social_Media	Sales
0	16.0	2.907983	54.732757
1	13.0	2.409567	46.677897
2	41.0	2.913410	150.177829
3	83.0	6.922304	298.246340
4	15.0	1.405998	56.594181
...	...	...	...
4567	26.0	0.717090	94.685866
4568	71.0	6.545573	249.101915
4569	44.0	5.096192	163.631457
4570	71.0	1.940873	253.610411
4571	42.0	5.046548	148.202414

4517 rows × 3 columns

Next steps: [Generate code with df_sales](#)[New interactive sheet](#)

1.3. ML Model Development

```
#feature matrix  
X = df_sales.drop(columns="Sales")  
#target vector  
y = df_sales["Sales"]
```

Start coding or generate with AI.